Microprocessors and Microcomputers: A Comprehensive Overview

Microprocessor Internals

A microprocessor is the central processing unit (CPU) of a computer or embedded system. It is a complex integrated circuit (IC) responsible for executing a sequence of instructions, performing arithmetic and logic operations, controlling data flow, and interacting with peripheral devices. Understanding the internals of a microprocessor involves looking into its key components and how they work together to perform computational tasks.

1. Basic Structure of a Microprocessor

The basic structure of a microprocessor consists of several key functional blocks:

- **Control Unit (CU)**
- **Arithmetic Logic Unit (ALU)**
- **Registers**
- **Bus System**
- **Clock**
- **Cache Memory**
- **Instruction Set Architecture (ISA)**

1.1 Control Unit (CU)

The control unit is responsible for directing the operation of the processor. It fetches, decodes, and executes instructions in the correct sequence. The control unit generates control signals that regulate data flow between the various components within the microprocessor.

Key tasks of the CU:

- Fetches the instruction from memory.
- Decodes the instruction to determine which operation to perform.
- Coordinates data movement between the registers, ALU, and memory.
- Controls input and output (I/O) operations.

1.2 Arithmetic Logic Unit (ALU)

The ALU performs mathematical calculations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT, etc.). It is a key component for performing any computational task in the processor.

Key tasks of the ALU:

- Executes arithmetic operations like addition, subtraction, multiplication, and division.
- Performs logical operations like AND, OR, XOR, and NOT.
- May also include comparison operations like less than, greater than, and equality.

1.3 Registers

Registers are small, high-speed storage locations within the processor. They temporarily hold data that the processor needs for immediate use, such as operands for arithmetic operations or addresses for memory locations. Registers vary in function and size based on the architecture.

Common types of registers:

- **General-purpose registers (GPRs):** Used for temporary data storage during computation.
- **Special-purpose registers:**
    - **Program Counter (PC):** Holds the address of the next instruction to be fetched.
    - **Instruction Register (IR):** Holds the current instruction being executed.
    - **Status Register/Flags Register:** Stores status flags that indicate the result of an operation, such as carry, zero, overflow, or negative results.

1.4 Bus System

The bus system is a collection of electrical paths that transfer data and control signals between different parts of the microprocessor and between the processor and memory. The bus system can be divided into:

- **Data Bus:** Transfers data between components.
- **Address Bus:** Carries memory addresses to access data in memory.
- **Control Bus:** Carries control signals to synchronize operations.

The width of the bus (number of bits it can carry simultaneously) impacts the processor's overall performance.

1.5 Clock

The clock is responsible for synchronizing the timing of all the operations within the microprocessor. It generates a series of pulses, or clock cycles, that control the rate at which instructions are executed. The clock speed is typically measured in gigahertz (GHz) and influences the speed at which a microprocessor can process instructions.

1.6 Cache Memory

Cache memory is a small, high-speed memory located closer to the processor core. It stores frequently accessed data and instructions to reduce the time it takes for the CPU to fetch data from main memory (RAM). There are typically multiple levels of cache:

- **L1 Cache:** Smallest and fastest, integrated into the CPU.
- **L2 Cache:** Larger but slower than L1, may be integrated into the CPU or located externally.
- **L3 Cache:** Largest cache, typically shared between multiple cores in a multi-core processor.

1.7 Instruction Set Architecture (ISA)

The ISA defines the set of instructions the microprocessor can execute and the corresponding machine code. It serves as an interface between the hardware and software. Different microprocessors may implement different ISAs, which determines the type of instructions the processor can handle.

Common ISAs:

- **x86 (Intel/AMD):** A widely used ISA for personal computers.
- **ARM:** An ISA designed for low-power, mobile devices and embedded systems.
- **MIPS:** An ISA used in embedded and networking devices.

2. Microprocessor Operation Cycle

The operation of a microprocessor follows a cycle known as the **Fetch-Decode-Execute** cycle.

1. **Fetch:** The instruction is fetched from memory (RAM) using the Program Counter (PC), and it is loaded into the Instruction Register (IR).
2. **Decode:** The control unit decodes the instruction in the IR to understand which operation is to be performed and prepares the necessary resources (e.g., ALU, registers).
3. **Execute:** The ALU performs the necessary arithmetic or logical operation, or the control unit initiates memory read/write or I/O operations. The result is stored in registers or memory as needed.

This cycle repeats continuously until the program finishes or the processor is powered off.

3. Types of Microprocessors

Microprocessors come in different types based on the architecture and intended application. Some of the common types include:

- **Single-Core Processors:** Contain only one processing unit (core) that executes instructions sequentially.
- **Multi-Core Processors:** Contain multiple processing units (cores) that allow parallel processing, increasing performance for multi-threaded applications.
- **RISC (Reduced Instruction Set Computing) Processors:** Focus on a small, highly optimized set of instructions, which typically results in higher performance.
- **CISC (Complex Instruction Set Computing) Processors:** Have a large set of complex instructions, which can execute multiple operations in a single instruction.
- **DSP (Digital Signal Processor):** Specializes in high-speed numerical calculations, particularly for audio, video, and telecommunications.
- **Embedded Processors:** Designed for specific tasks, embedded in devices like appliances, cars, and medical equipment.

4. Advanced Microprocessor Features

- **Pipelining:** A technique where multiple instructions are processed in overlapping stages. It improves CPU performance by utilizing the processor's resources more efficiently.

- **Superscalar Architecture:** Allows the processor to execute more than one instruction per clock cycle by using multiple execution units.
- **Out-of-Order Execution:** Instructions are executed as resources become available, rather than strictly in program order, improving performance.
- **Hyper-Threading (Simultaneous Multi-Threading):** A technique where a single core acts as if it is multiple cores by allowing it to execute two threads simultaneously, improving parallelism.

5. Performance Considerations

When evaluating microprocessor performance, several factors come into play:

- **Clock Speed:** A higher clock speed means the processor can execute more instructions per second, but it also depends on the processor's efficiency.
- **Instruction Set Efficiency:** The complexity and design of the instruction set can affect how quickly a processor can execute tasks.
- **Core Count:** More cores allow for better multitasking and parallel processing.
- **Cache Size and Hierarchy:** A larger cache can reduce memory latency and improve performance.
- **Power Efficiency:** For mobile and embedded systems, power consumption is a key factor in processor design.

The internals of a microprocessor involve a complex interplay of various components like the ALU, CU, registers, and buses, each with distinct roles in processing instructions. A well-designed microprocessor can efficiently handle tasks by executing operations at high speed, managing power consumption, and utilizing parallelism where possible. As technology advances, modern processors incorporate features like multi-core architecture, pipelining, and hyper-threading to push computational performance even further.

Evolution of Microcomputers

1.1 Introduction to Microcomputers

Microcomputers are small, affordable computing systems that emerged in the mid-20th century. They played a crucial role in the widespread adoption of personal computing. They typically consist of a microprocessor, memory, storage, and input/output peripherals.

Microcomputers contrast with mainframes and minicomputers, which were larger and more expensive. They became popular due to their affordability, ease of use, and capability to run general-purpose applications.

1.2 Evolution of Microcomputers

I. **4-bit Microprocessors :** The first microprocessor was introduced in 1971 by Intel Corp. It was named Intel 4004 as it was a 4 bit processor. It was a processor on a single chip. It could

perform simple arithmetic and logic operations such as addition, subtraction, boolean AND and boolean OR. Intel introduced the enhanced version of 4004, the 4040.

II. **8-bit Microprocessors:** The first 8 bit microprocessor which could perform arithmetic and logic operations on 8 bit words was introduced in 1973 again by Intel. This was Intel 8008 and was later followed by an improved version, Intel 8088. 16-bit Microprocessors The 8-bit processors were followed by 16 bit processors. They are Intel 8086 and 80286.

III. **32-bit Microprocessors:** The 32 bit microprocessors were introduced by several companies but the most popular one is Intel 80386.

IV. **Pentium Series:** Instead of 80586, Intel came out with a new processor namely Pentium processor. Its performance is closer to RISC performance. Pentium was followed by Pentium Pro CPU. Pentium Pro allows allow multiple CPUs in a single system in order to achieve multiprocessing. The MMX extension was added to Pentium Pro and the result was Pentiuum II. The low cost version of Pentium II is celeron. The Pentium III provided high performance floating point operations for certain types of computations by using the SIMD extensions to the instruction set. These new instructions makes the Pentium III faster than high-end RISC CPUs. Interestingly Pentium IV could not execute code faster than the Pentium III when running at the same clock frequency. So Pentium IV had to speed up by executing at a much higher clock frequency.

NAME	YEAR	TRANSISTORS	DATA WIDTH	CLOCK SPEED
8080	1974	6,000	8 bits	2 MHz
8085	1976	6,500	8 bits	5 MHz
8086	1978	29,000	16 bits	5 MHz
8088	1979	29,000	8 bits	5 MHz
80286	1982	134,000	16 bits	6 MHz
80386	1985	275,000	32 bits	16 MHz
80486	1989	1,200,000	32 bits	25 MHz
PENTIUM	1993	3,100,000	32/64 bits	60 MHz
PENTIUM II	1997	7,500,000	64 bits	233 MHz
PENTIUM III	1999	9,500,000	64 bits	450 MHz
PENTIUM IV	2000	42,000,000	64 bits	1.5 GHz

The evolution of microcomputers has significantly impacted industries such as healthcare, automation, finance, and entertainment. From early 8-bit processors to modern multi-core architectures, microcomputers have become an essential part of everyday life and technological advancement.

Microcomputer Applications

1. Business Applications

Microcomputers play a crucial role in modern business operations, streamlining processes and improving productivity. Key business applications include:

- **Word Processing**: Software like Microsoft Word and Google Docs is used for document creation, editing, and formatting.
- **Spreadsheets**: Tools like Microsoft Excel and Google Sheets are used for financial analysis, data management, and reporting.
- **Database Management**: Applications such as Microsoft Access, MySQL, and SQLite store and manage large amounts of structured data.
- **Accounting and Financial Management**: Software like QuickBooks, Tally, and FreshBooks helps businesses manage accounts, invoices, and payroll.
- **Presentation Software**: Microsoft PowerPoint and Google Slides assist in creating professional presentations.
- **Enterprise Resource Planning (ERP)**: Systems like SAP and Oracle streamline business processes across various departments.

2. Educational Applications

Microcomputers enhance learning experiences and improve knowledge delivery. Key educational applications include:

- **E-Learning Platforms**: Moodle, Blackboard, and Coursera facilitate online education and training.
- **Computer-Aided Instruction (CAI)**: Interactive software helps students understand complex subjects through simulations and multimedia.
- **Virtual Labs**: Applications like PhET and Labster allow students to conduct virtual experiments.
- **Typing Tutors**: Software like TypingMaster helps improve keyboarding skills.
- **Student Information Systems**: Platforms like PowerSchool and Infinite Campus manage student records and academic performance.

3. Healthcare Applications

Microcomputers improve healthcare services by managing patient records, diagnosing diseases, and supporting medical research. Applications include:

- **Electronic Health Records (EHR)**: Systems like Epic and Cerner store and manage patient medical history.
- **Medical Imaging**: Software like DICOM and PACS processes X-rays, MRIs, and CT scans.

- **Telemedicine**: Applications like Zoom for Healthcare and Teladoc facilitate remote consultations.
- **Medical Research and Simulation**: Programs like MATLAB and SPSS assist in medical data analysis and research.

4. Entertainment Applications

Microcomputers are widely used in entertainment industries, including gaming, music, and video production. Examples include:

- **Multimedia Production**: Adobe Photoshop, Premiere Pro, and Audacity help create images, videos, and audio content.
- **Gaming**: Platforms like Steam and Epic Games provide access to a vast collection of video games.
- **Streaming Services**: Netflix, YouTube, and Spotify offer on-demand entertainment.
- **Animation and 3D Modeling**: Software like Blender and Autodesk Maya assist in animation and CGI creation.

5. Scientific and Engineering Applications

Microcomputers are extensively used for computational tasks, data analysis, and simulations in scientific and engineering fields. Key applications include:

- **Computer-Aided Design (CAD)**: AutoCAD and SolidWorks assist in engineering design.
- **Statistical and Data Analysis**: SPSS, R, and Python help in processing and analyzing complex data.
- **Mathematical Computing**: MATLAB and Mathematica perform complex mathematical operations and simulations.
- **Geographic Information Systems (GIS)**: Software like ArcGIS and QGIS is used for mapping and spatial analysis.

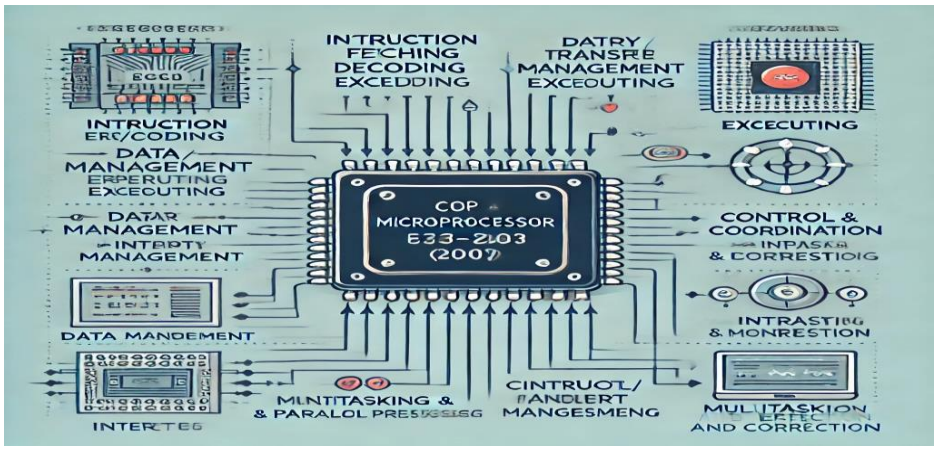
6. Security and Networking Applications

Security and networking tools ensure safe and efficient data communication. Important applications include:

- **Antivirus and Security Software**: Norton, McAfee, and Bitdefender protect against malware and cyber threats.
- **Firewalls and VPNs**: Cisco Firewall and NordVPN enhance network security and privacy.
- **Network Monitoring**: Wireshark and PRTG Network Monitor help diagnose and monitor network performance.
- **Encryption Tools**: PGP and VeraCrypt protect sensitive data through encryption.

Tasks of a Microprocessor

A microprocessor is the central processing unit (CPU) of a computer or an embedded system, responsible for executing instructions and managing computational tasks. It serves as the brain of the system, handling various operations to ensure efficient processing. Below are the primary tasks performed by a microprocessor:



1. Instruction Fetching

The microprocessor retrieves instructions from memory. This is the first step in executing a program, where instructions stored in RAM or ROM are accessed via the system bus.

2. Instruction Decoding

Once the instruction is fetched, the microprocessor decodes it to determine the operation that needs to be performed. The instruction decoder converts the binary code into control signals that drive internal operations.

3. Execution of Instructions

The microprocessor executes the decoded instructions using its arithmetic logic unit (ALU), registers, and control unit. This may involve arithmetic operations (addition, subtraction, multiplication, division), logical operations (AND, OR, NOT), or data movement (load, store, transfer).

4. Memory Management

The microprocessor manages access to memory, reading data from or writing data to RAM or cache. It also handles memory addressing techniques such as direct, indirect, or indexed addressing to efficiently locate data.

5. Data Transfer

The microprocessor facilitates communication between different components of the system by moving data between registers, memory, and input/output (I/O) devices.

6. Input/Output Operations

The microprocessor interacts with peripheral devices such as keyboards, displays, and storage devices. It processes input signals, interprets them, and provides the necessary output through buses and ports.

7. Control and Coordination

It acts as the control center of the system, coordinating the activities of all components. The control unit generates timing and control signals to synchronize the execution of instructions.

8. Interrupt Handling

The microprocessor responds to external or internal interrupts, pausing its current operations to handle higher-priority tasks, such as responding to user input or managing real-time system events.

9. Power Management

In modern systems, microprocessors manage power consumption through techniques such as clock gating, dynamic voltage scaling, and sleep modes to optimize energy efficiency.

10. Multitasking and Parallel Processing

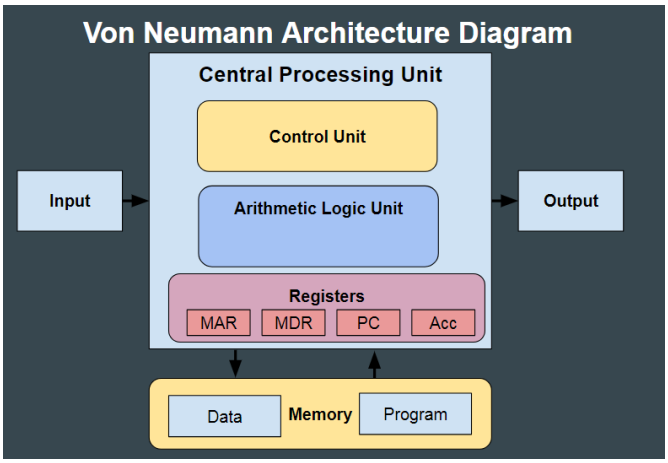
Advanced microprocessors support multitasking and parallel processing, allowing multiple processes to execute concurrently using multiple cores or threads.

11. Error Detection and Correction

Some microprocessors implement error detection and correction mechanisms, such as parity checks and error-correcting codes (ECC), to ensure data integrity and reliable operation.

The Von Neumann Model

The Von Neumann model, also known as the Von Neumann architecture, is a fundamental computer architecture concept that has profoundly influenced the design of modern microprocessors and microcomputers. It was first proposed by the Hungarian mathematician and physicist John Von Neumann in the mid-20th century and is still the basis for most contemporary computing systems.



1. Introduction

The Von Neumann model serves as the blueprint for the construction of digital computers. It describes a system where the memory is used to store both instructions and data, which is processed sequentially by the computer's control unit. This architecture contrasts with the Harvard architecture, where instructions and data are stored separately.

2. Key Components of the Von Neumann Architecture

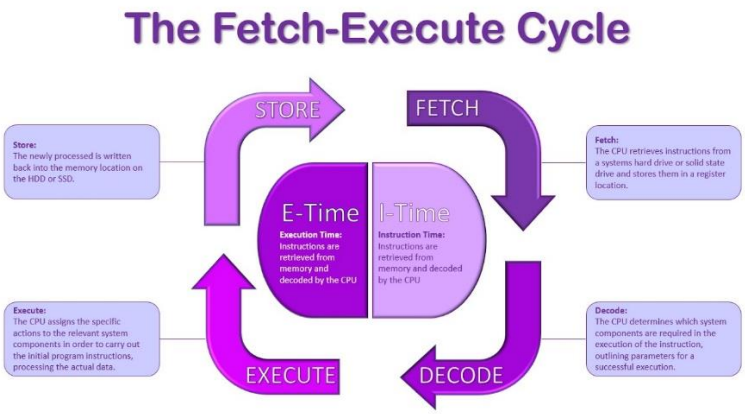
The Von Neumann architecture is composed of the following core components:

- **Central Processing Unit (CPU)**:
  - The CPU is the heart of the computer, responsible for executing instructions. It consists of two primary subcomponents:
    - **Arithmetic Logic Unit (ALU)**: Handles arithmetic and logical operations, such as addition, subtraction, and comparisons.
    - **Control Unit (CU)**: Directs the operation of the processor by interpreting and executing instructions fetched from memory.
- **Memory**:
  - **Primary Memory (RAM)**: Stores both data and program instructions. The memory in Von Neumann architecture is often referred to as a **unified memory system** because it handles both program code and data in the same memory space.
  - **Secondary Memory**: Non-volatile storage devices, like hard drives or SSDs, are used for persistent data storage but are not part of the Von Neumann architecture itself.
- **Input/Output Devices (I/O)**:
  - These are the external peripherals such as keyboards, monitors, and network interfaces that allow the computer to interact with the outside world.
- **Buses**:
  - A bus is a communication pathway used to transfer data between components of the system. In Von Neumann architecture, there are typically three buses:
    - **Data Bus**: Carries the actual data being transferred between components.
    - **Address Bus**: Carries the memory addresses of the data being accessed.

- **Control Bus**: Carries signals that manage the operations of the computer, including read/write commands and timing signals.

3. How the Von Neumann Architecture Works

The operation of a Von Neumann system follows a specific cycle of steps known as the **fetch-decode-execute cycle**, which is repeated continually while the computer is running.

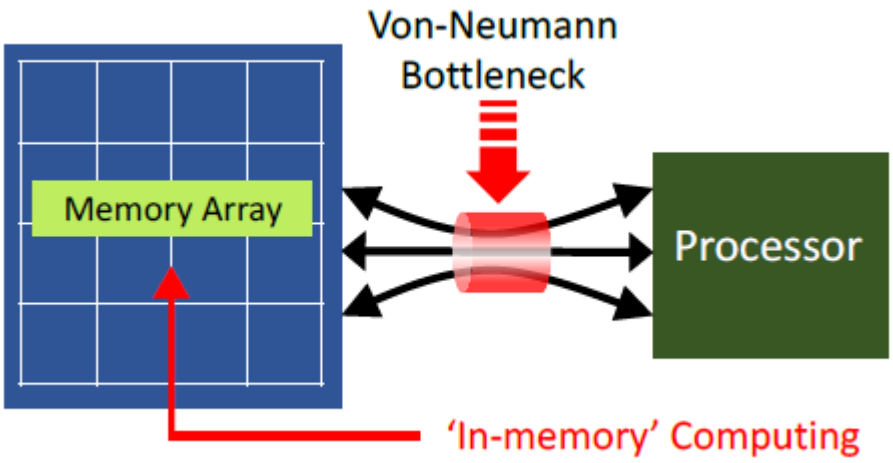


1. **Fetch**:
   - The **Control Unit (CU)** fetches the instruction from memory (RAM) at the address indicated by the Program Counter (PC).
2. **Decode**:
   - The fetched instruction is decoded to understand what operation needs to be performed. The CU interprets the opcode (operation code) and prepares the necessary data.
3. **Execute**:
   - The **ALU** performs the operation (arithmetic, logical, or data transfer) on the data in the registers or memory.
4. **Store**:
   - If necessary, the result of the execution is stored back into memory or a register for later use.

These operations are repeated continuously, and the program counter (PC) increments after each instruction to point to the next instruction in memory.

4. Von Neumann Bottleneck

One of the primary drawbacks of the Von Neumann architecture is the **Von Neumann bottleneck**. This issue arises because both program instructions and data share the same memory and system bus. As a result, the CPU often has to wait for data or instructions to be fetched from memory, which can limit performance. This is particularly noticeable when working with large datasets or complex programs. To mitigate this issue, modern systems incorporate **cache memory** (L1, L2, and L3 caches) to store frequently accessed data and instructions, reducing the delay caused by memory fetching.



The von Neumann bottleneck (VNB) is a performance issue in computers that limits how quickly they can operate. It's caused by the separation of memory and computing units, which increases the time and energy it takes to move data between them.

Here are some solutions to the von Neumann bottleneck:

I. **Cache memory**: Place a cache between the CPU and main memory to store data and instructions that are expected to be used soon
II. **Multithreading**: Allow the processor to manage multiple requests at once by switching between threads
III. **Prefetching**: Fetch data and instructions into the cache before they're needed
IV. **Speculative execution**: Use branch prediction to estimate which instructions will be needed next, and perform them before they're requested
V. **Near-data processing**: Add processing capabilities to memory and storage to reduce the need for the system bus
VI. **Hardware acceleration**: Shift processing to other hardware devices to reduce the load on the CPU
VII. **System-on-a-chip**: Integrate the CPU, memory, and other system resources into a single chip
VIII. **Processing-in-memory (PIM)**: Minimize data movement between memory hierarchies
IX. **Reconfigurable systems**: Design systems that can be reconfigured to improve performance

5. The Evolution of Von Neumann in Microprocessors

Microprocessors are designed based on the Von Neumann model, although many modern processors implement modifications and optimizations to enhance performance. Key developments in Von Neumann-based microprocessor architecture include:

- **Pipelining**:
  - Modern CPUs use pipelining to improve instruction throughput. Pipelining allows different stages of instruction fetching, decoding, and execution to occur simultaneously, increasing the overall speed of the processor.
- **Superscalar Architecture**:
  - This approach allows the CPU to execute multiple instructions per clock cycle, improving processing efficiency.
- **Parallel Processing**:
  - Some modern processors incorporate multi-core designs, where multiple independent CPUs (cores) work in parallel, further enhancing computational performance.
- **Out-of-Order Execution**:
  - This technique allows processors to execute instructions in a non-sequential order based on the availability of resources, improving efficiency and minimizing idle time.

6. Von Neumann Architecture in Microcomputers

Microcomputers, such as personal computers, laptops, and even embedded systems, commonly use Von Neumann-based architectures, with modifications to improve performance. The architecture's simplicity and flexibility allow for a broad range of applications, from office tasks to complex computations.

In microcomputers:

- **Embedded Systems**: Many embedded devices use Von Neumann-based microcontrollers that are optimized for specific tasks, such as in robotics, automotive systems, and IoT (Internet of Things) devices.
- **Desktop Computers and Laptops**: These devices use advanced Von Neumann processors (such as Intel Core and AMD Ryzen chips) with multi-core architectures and complex memory hierarchies to maximize performance.

7. Benefits and Limitations of Von Neumann Architecture

Benefits:

- **Simplicity**: The architecture is straightforward and easy to implement, making it an ideal choice for general-purpose computing.
- **Flexibility**: Von Neumann systems can easily be adapted for a wide range of applications, from embedded systems to high-performance computing.
- **Cost-Effective**: The unified memory system reduces hardware complexity and cost.

Limitations:

- **Von Neumann Bottleneck**: The shared memory for both instructions and data can result in slower data transfer rates.

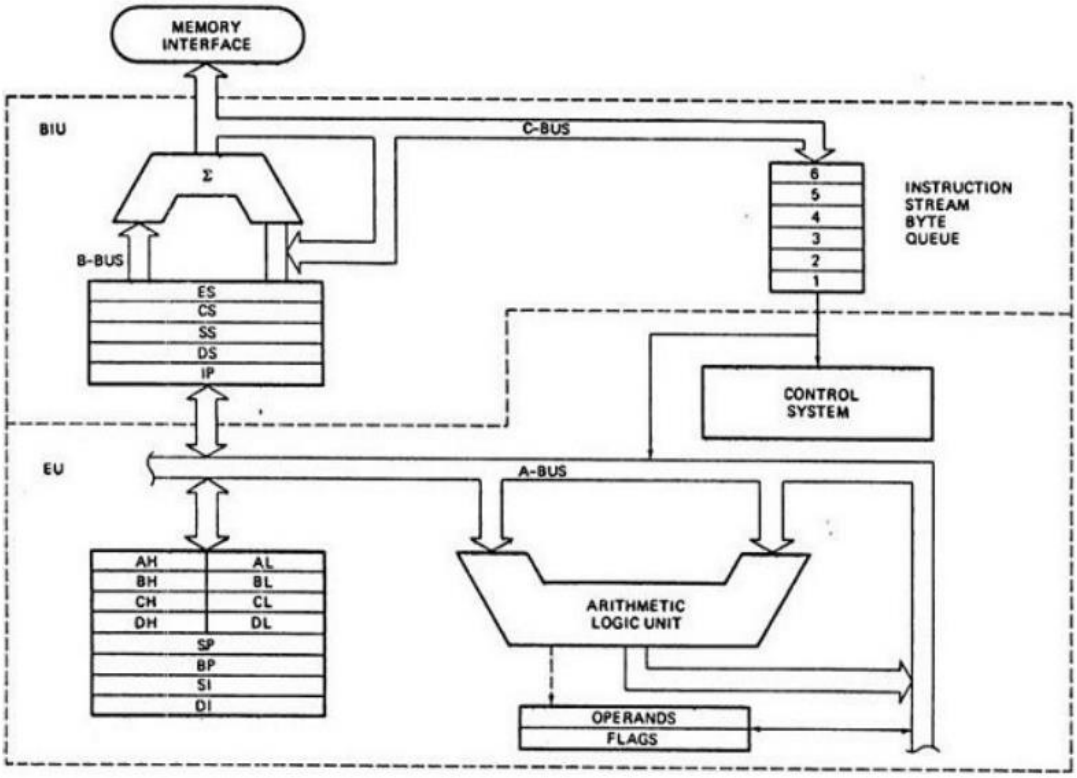
- **Sequential Execution**: The inherent sequential nature of the architecture can limit parallelism and performance in certain applications.

8086 Microprocessor

8086 Microprocessor is an enhanced version of 8085Microprocessor that was designed by Intel in 1976. It is a 16-bit Microprocessor having 20 address lines and16 data lines that provides up to 1MB storage. It consists of powerful instruction set, which provides operations like multiplication and division easily. It supports two modes of operation, i.e. Maximum mode and Minimum mode. Maximum mode is suitable for system having multiple processors and Minimum mode is suitable for system having a single processor.

Features of 8086 : The most prominent features of a 8086 microprocessor are as follows –

- It has a 20 bit address bus can access upto 2 20 memory location
- Support upto 64 KB I/O Ports
- Word size is 16 bits
- 40 pin dual in line package
- Address ranges from 00000H to FFFFFH
- Memory is addressable- every byte has a separate address
- It has an instruction queue, which is capable of storing six instruction bytes from the memory resulting in faster processing.
- It was the first 16-bit processor having 16-bit ALU, 16-bit registers, internal data bus, and 16-bit external data bus resulting in faster processing.
- It is available in 3 versions based on the frequency of operation − o 8086 → 5MHz o 8086-2 → 8MHz o 8086-1 → 10 MHz
- It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage, which improves performance.
- Fetch stage can prefetch up to 6 bytes of instructions and stores them in the queue.
- Execute stage executes these instructions. It has 256 vectored interrupts.
- It consists of 29,000 transistors.



Architecture of 8086

The following diagram depicts the architecture of a 8086 Microprocessor – 8086 Microprocessor is divided into two functional units, i.e., EU (Execution Unit) and BIU (Bus Interface Unit).

BIU (Bus Interface Unit): BIU takes care of all data and addresses transfers on the buses for the EU like sending addresses, fetching instructions from the memory, reading data from the ports and the memory as well as writing data to the ports and the memory.

EU has no direction connection with System Buses so this is possible with the BIU. EU and BIU are connected with the Internal Bus. It has the following functional parts –

- Instruction queue − BIU contains the instruction queue. BIU gets upto 6 bytes of next instructions and stores them in the instruction queue. When EU executes instructions and is ready for its next instruction, then it simply reads the instruction from this instruction queue resulting in increased execution speed.
- Fetching the next instruction while the current instruction executes is called pipelining.
- Segment register − BIU has 4 segment buses, i.e. CS, DS, SS& ES. It holds the addresses of instructions and data in memory, which are used by the processor to access memory locations. It also contains 1 pointer register IP, which holds the address of the next instruction to executed by the EU.
    - ✓ CS − It stands for Code Segment. It is used for addressing a memory location in the code segment of the memory, where the executable program is stored.

- ✓ DS − It stands for Data Segment. It consists of data used by the program and is accessed in the data segment by an offset address or the content of other register that holds the offset address.
- ✓ SS − It stands for Stack Segment. It handles memory to store data and addresses during execution.
- ✓ ES − It stands for Extra Segment. ES is additional data segment, which is used by the string to hold the extra destination data.

- Instruction pointer − It is a 16-bit register used to hold the address of the next instruction to be executed.

BIU contains a separate adder to perform physical address calculation.

EU (Execution Unit): Execution unit gives instructions to BIU stating from where to fetch the data and then decode and execute those instructions. Its function is to control operations on data using the instruction decoder & ALU. EU has no direct connection with system buses as shown in the above figure, it performs operations over data through BIU.

- ALU: It handles all arithmetic and logical operations, like +, −, ×, /, OR, AND, NOT operations.
- Flag Register It is a 16-bit register that behaves like a flip-flop, i.e. it changes its status according to the result stored in the accumulator. It has 9 flags and they are divided into 2 groups − Conditional Flags and Control Flags.
- Conditional Flags It represents the result of the last arithmetic or logical instruction executed. Following is the list of conditional flags –
  - ✓ Carry flag − This flag indicates an overflow condition for arithmetic operations.
  - ✓ Auxiliary flag − When an operation is performed at ALU, it results in a carry/barrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), then this flag is set, i.e. carry given by D3 bit to D4 is AF flag. The processor uses this flag to perform binary to BCD conversion.
  - ✓ Parity flag − This flag is used to indicate the parity of the result, i.e. when the lower order 8-bits of the result contains even number of 1's, then the Parity Flag is set. For odd number of 1's, the Parity Flag is reset.
  - ✓ Zero flag − This flag is set to 1 when the result of arithmetic or logical operation is zero else it is set to 0.
  - ✓ Sign flag − This flag holds the sign of the result, i.e. when the result of the operation is negative, then the sign flag is set to 1 else set to 0.
  - ✓ Overflow flag − This flag represents the result when the system capacity is exceeded.
- Control Flags: Control flags controls the operations of the execution unit. Following is the list of control flags –
  - ✓ Trap flag − It is used for single step control and allows the user to execute one instruction at a time for debugging. If it is set, then the program can be run in a single step mode.
  - ✓ Interrupt flag − It is an interrupt enable/disable flag, i.e. used to allow/prohibit the interruption of a program. It is set to 1 for interrupt enabled condition and set to 0 for interrupt disabled condition.

- ✓ Direction flag − It is used in string operation. As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-aversa.

General purpose register: There are 8 general purpose registers, i.e., AH, AL, BH, BL, CH, CL, DH, and DL. These registers can be used individually to store 8-bit data and can be used in pairs to store 16bit data. The valid register pairs are AH and AL, BH and BL, CH and CL, and DH and DL. It is referred to the AX, BX, CX, and DX respectively.

- ➢ AX register − It is also known as accumulator register. It is used to store operands for arithmetic operations.
- ➢ BX register − It is used as a base register. It is used to store the starting base address of the memory area within the data segment.
- ➢ CX register − It is referred to as counter. It is used in loop instruction to store the loop counter.
- ➢ DX register − This register is used to hold I/O port address for I/O instruction.
- ➢ Stack pointer register It is a 16-bit register, which holds the address from the start of the segment to the memory location, where a word was most recently stored on the stack.